COURSE CODE/TITLE: CPE 010-CPE21S1 - Data Structures and Algorithms	**SECTION: CPE21S1**
DATE PERFORMED: Oct 1, 2026	**DATE SUBMITTED: Oct 1, 2026**
NAME: Mendoza, Roy Gabriel I.	**INSTRUCTOR: Engr. Jimlord Quejado**

ACTIVITY NO. 10.1 Graphs

1. PROCEDURE OUTPUT

ILO A:

HEADER

```
#ifndef UNTITLED1_JIRAPH_H
#define UNTITLED1_JIRAPH_H

#include <iostream>
#include <vector>
#include <stack>
#include <queue>
#include <set>
#include <map>
//ILO A: Graph Implementation (Adjacency List & Adjacency Matrix)
template <typename T>
struct AdjNode {
    T val;
    T cost;
    AdjNode<T>* next;
};

template <typename T>
struct GraphEdge {
    int start_ver;
    int end_ver;
    T weight;
};

template <typename T>
class DiaGraph {
private:
    int N;

```

```
    AdjNode<T>* getAdjListNode(T value, T weight, AdjNode<T>* head) {
        AdjNode<T>* newNode = new AdjNode<T>;
        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head;
        return newNode;
    }

public:
    AdjNode<T>** head;

    DiaGraph(GraphEdge<T> edges[], int n, int N) {
        head = new AdjNode<T>*[N]();
        this->N = N;

        for (int i = 0; i < N; ++i)
            head[i] = nullptr;

        for (int i = 0; i < n; i++) {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            T weight = edges[i].weight;

            AdjNode<T>* newNode = getAdjListNode( value: static_cast<T>(end_ver), weight, head[start_ver]);
            head[start_ver] = newNode;
        }
    }
```

```
    /*
    // ORIGINAL DESTRUCTOR
    ~DiaGraph() {
        for (int i = 0; i < N; i++)
            delete[] head[i];
        delete[] head;
    }
    */

    ~DiaGraph() {
        for (int i = 0; i < N; i++) {
            AdjNode<T>* current = head[i];
            while (current != nullptr) {
                AdjNode<T>* next = current->next;
                delete current;
                current = next;
            }
        }
        delete[] head;
    }

    // Copying would make two objects free the same nodes (double free)
    DiaGraph(const DiaGraph&) = delete;
    DiaGraph& operator=(const DiaGraph&) = delete;
```

```
    // Copying would make two objects free the same nodes (double free)
    DiaGraph(const DiaGraph&) = delete;
    DiaGraph& operator=(const DiaGraph&) = delete;

    void display_AdjList() const {
        std::cout << "Graph adjacency list\n(start_vertex, end_vertex, weight):\n";
        for (int i = 0; i < N; i++) {
            AdjNode<T>* ptr = head[i];
            while (ptr != nullptr) {
                std::cout << "(" << i << ", " << ptr->val << ", " << ptr->cost << ") ";
                ptr = ptr->next;
            }
            std::cout << std::endl;
        }
    }
};

// ILO A.2: Adjacency Matrix Representation
template <typename T>
class AdjMatrixGraph {
private:
    int V;
    std::vector<std::vector<T>> matrix;
```

```
public:
    AdjMatrixGraph(int vertices) : V(vertices), matrix(vertices, std::vector<T>(vertices, 0)) {}

    void add_edge(int src, int dest, T weight = 1) {
        if (src >= 0 && src < V && dest >= 0 && dest < V) {
            matrix[src][dest] = weight;
        }
    }

    void display_matrix() const {
        std::cout << "Adjacency Matrix:\n";
        for (int i = 0; i < V; ++i) {
            for (int j = 0; j < V; ++j) {
                std::cout << matrix[i][j] << " ";
            }
            std::cout << "\n";
        }
    }
};
```

MAIN

```
#include <iostream>
#include "jiraph.h"

void test_ILO_A() {
    std::cout << "      ILO A: Adjacency List Output      \n";

    GraphEdge<int> edges[] = {
        { start_ver: 0,  end_ver: 1,  weight: 2}, { start_ver: 0,  end_ver: 2,  weight: 4}, { start_ver: 1,  end_ver: 4,  weight: 3}, { start_ver: 2,  end_ver: 3,  weight: 2}, { start_v
    };
    int N = 6;
    int n = sizeof(edges) / sizeof(edges[0]);

    DiaGraph<int> diagraph(edges, n, N);
    diagraph.display_AdjList();
    std::cout << std::endl;
}
```

```
46  int main() {
47      using T = unsigned;
48
49      // Run tests
50      test_ILO_A();
```

OUTPUT

```
      ILO A: Adjacency List Output
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

ANALYSIS

DiaGraph stores a directed, weighted graph as an array of N head pointers, where each head starts a singly linked list of AdjNode entries holding the destination vertex, the edge cost and a next pointer. Each edge is inserted at the head of its list in constant time, which is why vertex 0 prints (0, 2, 4) (0, 1, 2) with the neighbors in reverse insertion order. Vertex 5 has no edges, so nothing is printed for it. The destructor walks every list and deletes each node, because the sheet's delete[] head[i] was invalid for single nodes.

ILO B1

HEADER	

```cpp
//ILO B: Graph Traversal Algorithms (Edge-List Representation)
template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const {
        return this->weight > e.weight;
    }
};

template <typename T>
class Graph {
private:
    size_t V;
    std::vector<Edge<T>> edge_list;

public:
    Graph(size_t N) : V(N) {}

    auto vertices() const ->size_t {
        return V;
    }

    auto &edges() const ->const vector<Edge<T>> & {
        return edge_list;
    }

```

```
159
160         void add_edge(Edge<T> &&e) {
161             if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V)
162                 edge_list.emplace_back(e);
163             else
164                 std::cerr << "Vertex out of bounds" << std::endl;
165         }
166
167         auto outgoing_edges(size_t v) const ->vector<Edge<T>> {
168             std::vector<Edge<T>> edges_from_v;
169             for (auto &e :const Edge<T> & : edge_list) {
170                 if (e.src == v)
171                     edges_from_v.emplace_back(e);
172             }
173             return edges_from_v;
174         }
175
176         template <typename U>
177         friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
178     };
179
180     template <typename T>
181     std::ostream &operator<<(std::ostream &os, const Graph<T> &G) {
182         // for (size_t i = 1; i < G.vertices(); i++) {
183         for (size_t i = 1; i <= G.vertices(); i++) {
184             os << i << ":\t";
185             auto edges :vector<Edge<T>> = G.outgoing_edges(i);
186             for (auto &e :Edge<T> & : edges)
187                 os << "{" << e.dest << ": " << e.weight << "}, ";
188             os << std::endl;
189         }
190         return os;
191     }
192
```

```cpp

// B.1. Depth-First Search
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t start_vertex) ->vector<unsigned long long> {
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    stack.push(start_vertex);
    while (!stack.empty()) {
        auto current_vertex :size_t  = stack.top();
        stack.pop();

        if (visited.find(current_vertex) == visited.end()) {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            auto edges :vector<Edge<T>>  = G.outgoing_edges(current_vertex);
            for (auto it = edges.rbegin(); it != edges.rend(); ++it) {
                if (visited.find(it->dest) == visited.end()) {
                    stack.push(it->dest);
                }
            }
        }
    }
    return visit_order;
}

```

MAIN

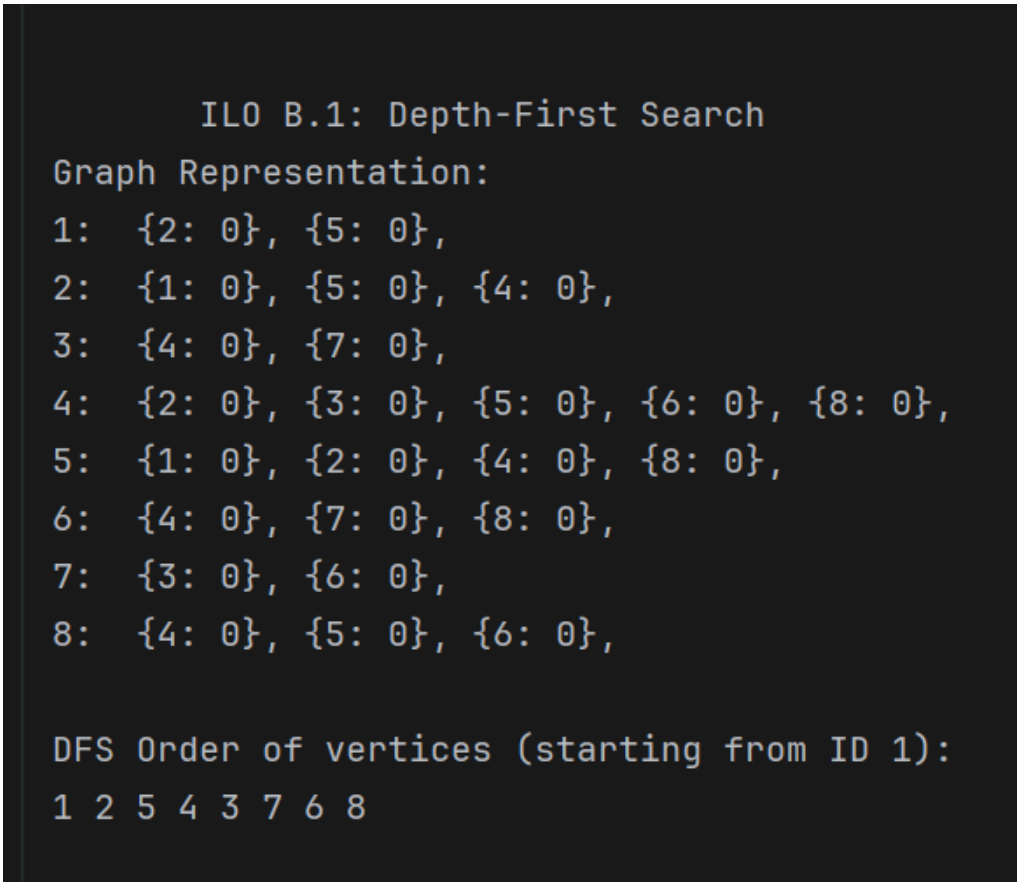
```cpp
template <typename T>
void test_DFS() {
    std::cout << "       ILO B.1: Depth-First Search      \n";

    auto G :Graph<T>  = create_reference_graph<T>();
    std::cout << "Graph Representation:\n" << G << std::endl;

    std::cout << "DFS Order of vertices (starting from ID 1):\n";
    auto dfs_visit_order :vector<size_t>  = depth_first_search(G,  start_vertex: 1);
    for (auto v :size_t  : dfs_visit_order)
        std::cout << v << " ";
    std::cout << "\n\n";
}

```

```cpp
test_DFS<T>();
```

OUTPUT	
ANALYSIS	The graph is stored as an edge list, so outgoing edges scans all edges on every call and costs O(E). The traversal uses a std::stack for pending vertices and a std::set for visited ones, and a vertex is marked when it is popped. Neighbors are pushed in reverse so that smaller vertex IDs are visited first. Each vertex is processed once, which gives a total cost of O(V·E). The output is 1 2 5 4 3 7 6 8.

ILO B2

HEADER

```
220  // B.2. Breadth-First Search
221  template <typename T>
222  auto breadth_first_search(const Graph<T> &G, size_t start_vertex) ->vector<unsigned long long> {
223      std::queue<size_t> queue;
224      std::vector<size_t> visit_order;
225      std::set<size_t> visited;
226
227      queue.push(start_vertex);
228      visited.insert(start_vertex);
229
230      while (!queue.empty()) {
231          auto current_vertex :size_t  = queue.front();
232          queue.pop();
233          visit_order.push_back(current_vertex);
234
235          /*
236          for (auto e : G.outgoing_edges(current_vertex))
237              queue.push(e.dest);
238          */
239          // for (auto e : G.outgoing_edges(current_vertex)) {
240          auto edges :vector<Edge<T>>  = G.outgoing_edges(current_vertex);
241          for (auto &e :Edge<T> &  : edges) {
242              if (visited.find(e.dest) == visited.end()) {
243                  visited.insert(e.dest);
244                  queue.push(e.dest);
245              }
246          }
247      }
248      return visit_order;
249  }
250
```

```cpp
template <typename T>
auto create_reference_graph() ->Graph<T> {
    // Graph<T> G(9);
    Graph<T> G(8);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};

    for (auto &i :pair<...> & : edges)
        for (auto &j :pair<unsigned long long, T> & : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}

#endif #ifndef UNTITLED1_JIRAPH_H
```

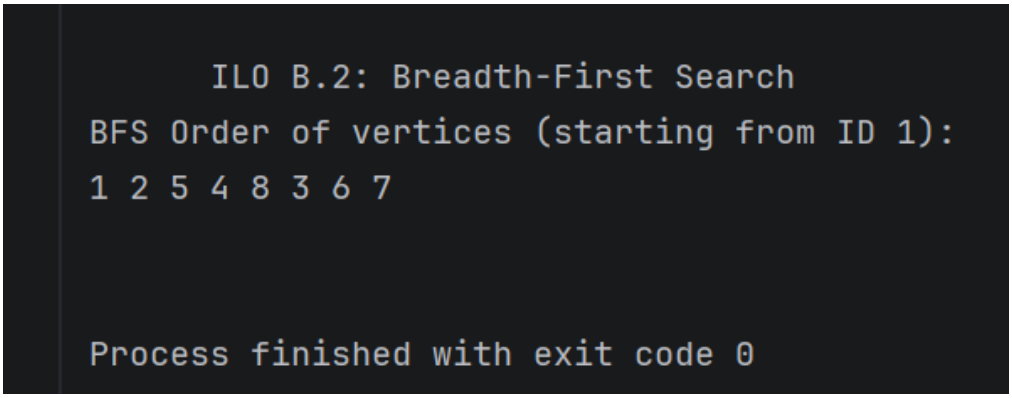
MAIN

```cpp
template <typename T>
void test_BFS() {
    std::cout << "      ILO B.2: Breadth-First Search     \n";

    auto G :Graph<T> = create_reference_graph<T>();

    std::cout << "BFS Order of vertices (starting from ID 1):\n";
    auto bfs_visit_order :vector<size_t> = breadth_first_search(G, start_vertex: 1);
    for (auto v :size_t : bfs_visit_order)
        std::cout << v << " ";
    std::cout << "\n\n";
}
```

```cpp
test_BFS<T>();
```

OUTPUT	
ANALYSIS	The traversal uses a **std::queue** and a **std::set,** and a vertex is marked as visited when it is enqueued, so it enters the queue only once. The vertices are visited level by level: {1}, {2, 5}, {4, 8}, {3, 6}, then {7}. The cost is O(V·E) for the same reason as DFS. The output is 1 2 5 4 8 3 6 7.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

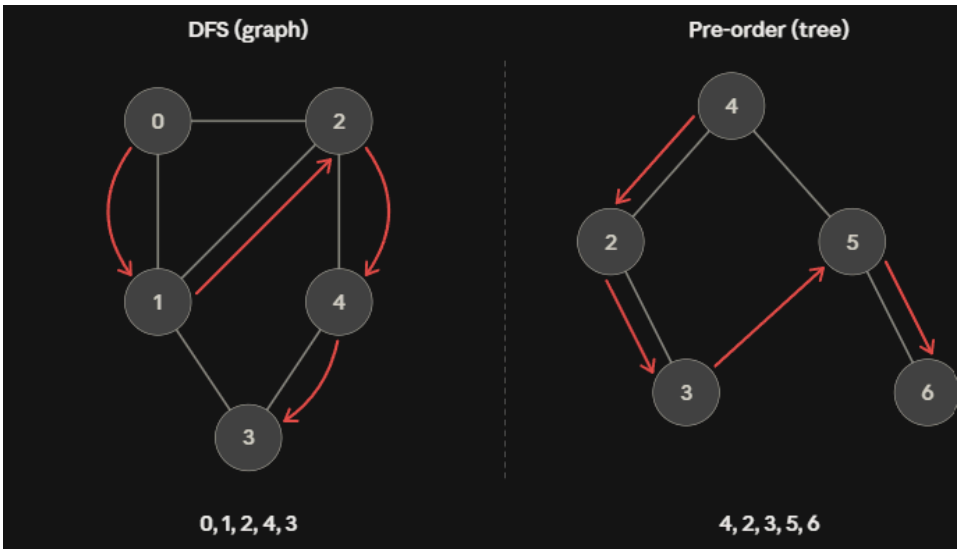
1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

3. In the performed code, what data structure is used to implement the Breadth First Search?

4. How many times can a node be visited in the BFS?

1. I would suggest the DFS because it follows a path maybe the left or right, then backtracks to the most recent vertex that is still unvisited, and then explore the neighbors from there.

2. 

   Both algorithms start at a single node, pre-order always starts at the root, while DFS can begin at any chosen vertex. They continuously move to the adjacent unvisited node with the smallest value, mark it as visited, and repeat the process from that node until every vertex in the graph or tree has been traversed.

3. A queue is the one that is used, there is also a code **<std::set<size_t> visited> to** help record which vertices have already been visited.

4. It can only be visited once, although it can be queued multiple times, it will still be visited once.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **None**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **None**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

